Локальная оптимизация геометрии в psiresp

Что выполнялось раньше и как работает исправленный путь в ligtools
Состояние локального кода на 13 ноября 2026 года

Главное изменение

Одиночный расчёт градиента заменён вызовом **psi4.optimize()**. Теперь координаты изменяются итерационно, а результат считается оптимизированным только после успешного завершения оптимизатора. Изменён локальный путь vendored-форка psiresp 0.4.2 в ligtools.

Часть расчёта	Раньше	Теперь
Входное задание	AtomicInput с driver="gradient"	OptimizationInput с исходной молекулой и спецификацией Psi4
Выполнение	Один расчёт градиента через psi4 --qcschema	Скрипт .opt.py вызывает psi4.optimize(); шагами управляет OptKing
Координаты результата	result.molecule.geometry — исходная геометрия одиночного расчёта	result.final_molecule.geometry — геометрия после оптимизации
Отметка is_optimized	Могла выставляться после одного градиента	Выставляется после чтения и проверки результата оптимизации
Старые файлы	Могли интерпретироваться как оптимизированные структуры	Исключены новым ключом кеша

Текущее поведение по умолчанию

Smi2Gmx использует геометрии MMFF94 без QM-оптимизации. Psi4 рассчитывает волновые функции при фиксированной геометрии. Для включения исправленной QM-оптимизации нужно явно передать optimize_geometry=True при создании Smi2Gmx.

Почему прежний путь был ошибочным

Градиент показывает направление изменения энергии, но сам по себе не перемещает атомы и не ищет минимум. В прежнем локальном пути объект AtomicResult возвращал геометрию одного расчёта; затем существующий Job.optimize_geometries() сохранял её и отмечал конформер как оптимизированный.

Область исправления

Исправлен локальный запуск Psi4. Существующий путь QCFractal отдельно отправлял задачу оптимизации через geometric; его алгоритм этим исправлением не заменялся. Метод расчёта зарядов остаётся двухстадийным RESP, HF/6-31G* в газовой фазе.

Как устроен новый путь

1. Подготовка задания

QMGeometryOptimizationOptions.write_input() записывает OptimizationInput: исходную молекулу, метод, базис и параметры оптимизатора. Создаётся вспомогательный файл .opt.py. Он вызывает локальный _optimization.py внутри интерпретатора выбранного исполняемого файла Psi4.

2. Настоящая оптимизация

```
energy, wavefunction, history = psi4.optimize(
    specification["model"]["method"],
    molecule=molecule,
    return_wfn=True, return_history=True,
)
```

OptKing выбирает перемещения атомов и проверяет сходимость, используя энергии и градиенты Psi4. Вложенный driver="gradient" сохранён: градиенты нужны на шагах оптимизации. Он больше не означает, что весь локальный этап ограничен одним градиентом.

3. Сохранение и принятие результата

```
final_molecule["geometry"] = np.asarray(
    wavefunction.molecule().geometry()
)
```

При успешном возврате psi4.optimize() сохраняются OptimizationResult, конечные координаты, история энергий и градиентов, число оценок в истории и диагностические величины градиента. Проверяется отсутствие NaN/Inf. Файл заменяется атомарно через временный файл.

Сходимость проверяет сам оптимизатор; обёртка не реализует независимую повторную проверку всех критериев. Она ставит маркер converged=True после нормального возврата optimize(). При исключении записывается success=False и ошибка передаётся вызывающему коду.

При чтении требуется OptimizationResult, маркер локального оптимизатора, подтверждение сходимости и final_molecule. Одиночный AtomicResult отклоняется. Ошибка процесса Psi4 также останавливает пайплайн. После принятия результата существующий Job переносит координаты в конформер и устанавливает is_optimized=True.

Кеш и последующие этапы

К хешу задания добавлена версия пути psi4-optimize-v1: прежние файлы градиентов не переиспользуются. Уже созданные в старом ядре Jupyter объекты нужно пересоздать. run_resp() теперь использует подготовленные ориентации и волновые функции, не запускает повторно генерацию ориентаций и QM.

Файлы, настройки и проверки

Файл	Что смотреть
psiresp/qm.py	write_input(), read_output(), _postprocess_result(), get_hash()
psiresp/_optimization.py	run_optimization(): psi4.optimize(), конечная геометрия, история и ошибки
smi2gmx.py	_run_psi4_input(): запуск .opt.py; run_resp(): использование готовых QM-данных
psiresp/job.py	optimize_geometries(): существующий перенос результата в конформеры
tests/test_local_optimization.py	Кеш, отклонение одиночного градиента, сходимость/несходимость и RESP

Настройки библиотеки и настройки прогона TPP

Параметр	По умолчанию в библиотеке	Прогон TPP с QM-оптимизацией
Метод / базис	HF / 6-31G*	HF / 6-31G*
max_iter	200	200
g_convergence (базовый набор)	gau_tight	gau_tight
convergence_tolerance	1e-3 для четырёх критериев	1e-3 (последняя попытка)
full_hess_every	10	−1: модельный гессиан с обновлениями

Значение −1 для TPP было задано только в скрипте прогона после остановки Psi4 системой из-за нехватки памяти при расчёте полного гессиана. Значение библиотеки осталось 10. Это изменение настройки оптимизации, а не включение нового протокола расчёта зарядов.

Проверки и текущий пайплайн

На искажённой молекуле воды при строгом gau_tight (convergence_tolerance=None) оптимизация HF/6-31G* прошла 12 шагов: энергия снизилась с −75,80612412 до −76,01072066 Hartree, конечный максимальный компонент градиента составил около 7,58 × 10^{-8} Hartree/bohr. Проверены изменение координат и повторное использование сошедшегося кеша. Искусственный лимит в один шаг привёл к ошибке, без ложного is_optimized=True.

Метанол прошёл всю цепочку в настоящем Jupyter: оптимизация → ESP → RESP → ACPYPE → экспорт файлов. Прогон TPP(−2) был остановлен пользователем после 38-го шага **без достижения сходимости**; он не подтверждает получение готовых параметров TPP.

При включении QM-оптимизации четыре явных допуска силы и перемещения равны 1e-3 в атомных единицах внутренних координат. convergence_tolerance=None возвращает пороги именованного набора. При линейных координатах это Hartree/bohr и bohr, при угловых — Hartree/рад и рад.

Отдельный полный прогон TPP(−2) на геометрии MMFF94, без QM-оптимизации, успешно прошёл до RTP в копии силового поля: 42 атома, заряд −2. Такой же полный прогон прошли API, MPO и MPI с зарядами −1. Порядок атомов, сохранность геометрии и заряды выходных файлов проверены.

Минимизация RDKit/MMFF94 — отдельный предварительный этап. Она включена по умолчанию перед окончательным отбором конформеров и не заменяет QM-оптимизацию. В CoordMatcher оптимизации геометрии нет.

Основание отчёта: локальные файлы репозитория и журналы ligtools-audit/local-optimization/ и ligtools-audit/tpp-minus2-full/. Описание относится к рабочей копии, а не к опубликованной версии GitHub.